Important ga missing Kubernetes interview topics

61. What is Kubernetes YAML Manifest?

A Kubernetes YAML Manifest is a configuration file used to define the desired state of a Kubernetes resource. Instead of creating resources manually using many commands, we can define the resource configuration in a YAML file and apply it to the Kubernetes cluster. A YAML file can define resources such as Pods, Deployments, Services, ConfigMaps, Secrets, and many others. The important fields commonly found in a Kubernetes manifest are apiVersion, kind, metadata, and spec. apiVersion tells Kubernetes which API version is being used, kind tells Kubernetes what type of resource we want to create, metadata contains information such as the resource name and namespace, and spec defines the desired configuration of that resource.
EXAMPLE:
Suppose our e-commerce application needs a Backend Deployment.
We can define:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3
```

Here:
apiVersion → API version
kind → Deployment
metadata.name → backend
spec.replicas → 3 Pods required
When we run:

```
kubectl apply -f backend.yaml
```

Kubernetes reads the YAML file and tries to create the desired state.
For example:
backend.yaml
↓
kubectl apply
↓
Kubernetes API Server
↓
Deployment
↓
ReplicaSet
↓
3 Pods
↓
Containers
The YAML file does not directly tell Kubernetes every individual step to perform. Instead, it describes what we want, and Kubernetes controllers work to make the actual cluster state match that desired configuration.
Simple flow:
YAML Manifest
↓
kubectl apply
↓
API Server
↓
Kubernetes Resource
↓
Desired State
↓

Actual State Managed
Main point:
Kubernetes YAML Manifest = A declarative configuration file used to define the desired state of Kubernetes resources.

62. What is kubectl?

kubectl is the command-line tool used to communicate with and manage a Kubernetes cluster. It sends requests to the Kubernetes API Server, and through these requests we can create, update, inspect, delete, and troubleshoot Kubernetes resources.

For example, as a DevOps engineer, we may use kubectl to check Pods, Deployments, Services, logs, events, and cluster resources.

EXAMPLE:

Suppose our e-commerce application is running in Kubernetes.

To check Pods:

```
kubectl get pods
```

To check Deployments:

```
kubectl get deployments
```

To check Services:

```
kubectl get svc
```

To see detailed information about a Pod:

```
kubectl describe pod backend-pod
```

To check application logs:

```
kubectl logs backend-pod
```

To create or update resources from a YAML file:

```
kubectl apply -f deployment.yaml
```

The important point is that kubectl does not directly manage the worker node. It communicates with the Kubernetes API Server, which is the central API interface of the cluster.

Simple flow:

```
DevOps Engineer
     ↓
   kubectl
     ↓
 API Server
     ↓
 Kubernetes Cluster
     ↓
 Pods / Services / Deployments
```

Main point:
kubectl = Command-line tool used to communicate with and manage Kubernetes clusters.

63. What is Pod Lifecycle in Kubernetes?

Pod Lifecycle describes the different stages a Pod goes through from creation until it is terminated. Kubernetes creates Pods based on the desired state, schedules them onto nodes, starts their containers, monitors them, and eventually terminates them when they are no longer required.

A Pod can have phases such as:

Pending
Running
Succeeded
Failed
Unknown

Pending means the Pod has been accepted by Kubernetes but is not yet running successfully, for example because it is waiting for scheduling, image pulling, or another required condition.

Running means the Pod has been scheduled and at least one container is running or starting.

Succeeded means all containers in the Pod have completed successfully.

Failed means all containers have terminated and at least one container failed.

Unknown means Kubernetes cannot determine the Pod's current state, often because communication with the node is unavailable.

EXAMPLE:
Suppose we deploy our e-commerce backend.
Initially:
Backend Pod
↓
Pending
Kubernetes schedules the Pod to a worker node.
Pending
↓
Scheduled
↓
Container Starting
↓
Running
If this is a Job and the task completes successfully:
Running
↓
Succeeded
If the application fails:
Running
↓
Failed
For a normal Deployment, Kubernetes usually creates replacement Pods when Pods terminate or fail, according to the Deployment's desired replica count.
Simple flow:
Pod Created
↓
Pending
↓
Running
↓
Succeeded / Failed
Main point:
Pod Lifecycle = The stages a Kubernetes Pod goes through from creation to termination.

---

64. What is Pod Restart Policy?
Pod Restart Policy defines how Kubernetes should handle containers that terminate inside a Pod. The available restart policies are Always, OnFailure, and Never.
Always means Kubernetes restarts terminated containers regardless of their exit status. This is the default behavior for Pods managed by controllers such as Deployments.
OnFailure means the container is restarted only when it terminates with a failure.
Never means Kubernetes does not restart the terminated container.
EXAMPLE:
Suppose our e-commerce backend is running continuously.
We want the application to keep running even if the container crashes.
For a Deployment, the Pod uses the appropriate restart behavior so Kubernetes can restart failed containers.
For a batch task such as database migration, we may want the container to run and finish instead of continuously restarting it.
For example:
Database Migration
↓
Container Runs
↓
Task Completed
↓
Pod Finished

This is why Jobs commonly use restart policies such as OnFailure or Never.
Simple flow:
Container Stops
↓
Restart Policy
↙ ↓ ↘
Always OnFailure Never
↓ ↓ ↓
Restart Failure Don't Restart
only
Main point:
Restart Policy = Defines when Kubernetes should restart containers inside a Pod.

---

65. What is a Headless Service in Kubernetes?
A Headless Service is a Kubernetes Service created with clusterIP: None. Unlike a normal ClusterIP Service, it does not provide a virtual cluster IP for load balancing. Instead, DNS can return the individual Pod IP addresses associated with the Service.
Headless Services are commonly useful for stateful applications where clients need to discover individual Pods rather than connect to one virtual Service IP.
EXAMPLE:
Suppose our e-commerce application uses a database cluster with:
database-0
database-1
database-2
The application may need to know the individual database instances.
We create:
Headless Service
↓
database-0
database-1
database-2
The DNS system can provide records that allow clients to discover these individual Pods.
This is commonly used with StatefulSets because StatefulSets provide stable Pod identities.
For example:
StatefulSet
↓
Headless Service
↓
database-0
database-1
database-2
Simple flow:
Application
↓
DNS
↓
Headless Service
↙ ↓ ↘
Pod 0 Pod 1 Pod 2
Main point:
Headless Service = A Service with no ClusterIP that provides DNS-based discovery of individual Pods.

---

66. What is CNI in Kubernetes?
CNI stands for Container Network Interface. It is a standard used by Kubernetes and container runtimes to configure networking for Pods. Kubernetes itself does not implement all Pod networking functionality. A CNI plugin is responsible for tasks such as assigning Pod IP addresses and configuring the network connectivity required for Pods.

Common CNI implementations include solutions such as Calico, Cilium, and Flannel.
EXAMPLE:
Suppose our e-commerce application has:
Frontend Pod
Backend Pod
Database Pod
When a new Pod is created, it needs network connectivity.
The CNI plugin helps configure that networking.
For example:
New Pod
↓
Container Runtime
↓
CNI Plugin
↓
Pod Network Configuration
↓
Pod Gets Network Connectivity
Now:
Frontend Pod
↓
Backend Pod
↓
Database Pod
can communicate according to the cluster's networking configuration and policies.
NetworkPolicy enforcement also depends on whether the selected networking implementation supports it.
Simple flow:
Pod Created
↓
CNI Plugin
↓
Network Interface
↓
Pod IP
↓
Pod Network Connectivity
Main point:
CNI = Provides the networking implementation used to connect Kubernetes Pods to the cluster network.

---

67. What is Container Runtime in Kubernetes?
A Container Runtime is the software responsible for running containers on Kubernetes worker nodes.
Kubernetes uses the Container Runtime Interface, or CRI, to communicate with a compatible container runtime.
A common modern Kubernetes runtime is containerd. Other CRI-compatible runtimes also exist.
The container runtime is responsible for operations such as pulling container images and creating, starting, stopping, and deleting containers.
EXAMPLE:
Suppose Kubernetes needs to run our backend container.
The flow is:
Kubernetes
↓
Kubelet
↓
CRI
↓
Container Runtime

↓
Container
↓
Backend Application

Suppose our Deployment specifies:

image: backend:v2

The runtime can pull the image if necessary and create the container according to the Pod specification.

Simple flow:

Pod Specification
↓
Kubelet
↓
CRI
↓
Container Runtime
↓
Container

Main point:

Container Runtime = Software on the worker node responsible for running containers.

---

68. What is CRI in Kubernetes?

CRI stands for Container Runtime Interface. It is the interface that allows Kubernetes components, particularly the Kubelet, to communicate with container runtimes.

Kubernetes does not need to directly understand the internal implementation of every container runtime. Instead, the Kubelet communicates through the CRI interface.

EXAMPLE:

Suppose our Kubernetes worker node uses containerd.

The communication can be represented as:

Kubelet
↓
CRI
↓
containerd
↓
Container

If another CRI-compatible runtime is used, Kubernetes can communicate with it through the same interface.

This provides an abstraction between Kubernetes and the container runtime.

Simple flow:

Kubernetes Kubelet
↓
CRI
↓
Container Runtime
↓
Containers

Main point:

CRI = Interface that allows Kubernetes Kubelet to communicate with container runtimes.

---

69. What is CNI vs CSI vs CRI?

CNI, CSI, and CRI are different Kubernetes-related interfaces, and understanding the difference is important in interviews.

CNI is related to networking.

CSI is related to storage.

CRI is related to container runtimes.

EXAMPLE:

Suppose our e-commerce backend Pod is created.

For networking:
Pod
↓
CNI
↓
Network
For storage:
Pod
↓
CSI
↓
Storage
For running containers:
Kubelet
↓
CRI
↓
Container Runtime
↓
Container
So each interface solves a different problem.
Simple comparison:
CNI → Networking
CSI → Storage
CRI → Container Runtime
Main point:
CNI = Network
CSI = Storage
CRI = Container Runtime

---

70. What is Pod Disruption Budget?

A PodDisruptionBudget, or PDB, is a Kubernetes resource used to limit how many replicated Pods of an application can be voluntarily disrupted at the same time.

It helps maintain application availability during operations such as node maintenance or cluster maintenance.

A PDB does not prevent every possible Pod failure. It mainly controls voluntary disruptions initiated through mechanisms that respect the PDB.

EXAMPLE:
Suppose our e-commerce backend has:
Backend → 5 replicas
We want at least 4 Pods to remain available during planned maintenance.
We can configure a PDB such as:
minAvailable: 4
Now Kubernetes-aware disruption processes should avoid voluntarily disrupting too many matching Pods at once.
For example:
5 Pods
↓
Maintenance
↓
1 Pod disrupted
↓
4 Pods remain available
This helps protect application availability during planned operations.
Simple flow:
5 Backend Pods
↓

PodDisruptionBudget
↓
Minimum Available Pods
↓
Controlled Voluntary Disruption
Main point:
PDB = Helps maintain a minimum level of application availability during voluntary Pod disruptions.

ChatGPT said:
Absolutely bro. 61 nunchi continue chestha. Already covered topics ni repeat cheyyanu. Important interview topics miss ayinavi, same style + clear example + enough depth lo istanu. E-commerce example ni continue chestha.
;;
61. What is Kubernetes Kubelet?
Kubelet is an agent that runs on every worker node in a Kubernetes cluster. Its main responsibility is to make sure that the Pods assigned to that node are running correctly according to their configuration. The Kubelet communicates with the Kubernetes API Server, receives information about Pods that should run on its node, and works with the container runtime to start and manage the containers. Kubelet does not decide which node a Pod should run on; that decision is made by the Scheduler.
EXAMPLE:
Suppose our e-commerce backend Deployment requires:
Backend → 3 Pods
The Scheduler decides:
Backend Pod 1 → Worker Node 1
Backend Pod 2 → Worker Node 2
Backend Pod 3 → Worker Node 3
Now Kubelet on Worker Node 1 is responsible for making sure Backend Pod 1 is actually running.
Worker Node 1
↓
Kubelet
↓
Container Runtime
↓
Backend Container
↓
Application
If the container stops, Kubelet detects the condition and works with the container runtime to restart it according to the Pod's configuration.
Kubelet also performs health checks such as configured liveness, readiness, and startup probes and reports Pod status back to the control plane.
Simple flow:
API Server
↓
Kubelet
↓
Container Runtime
↓
Container
↓
Application
Main point:
Kubelet = Worker-node agent that ensures assigned Pods and their containers are running as specified.
;;
62. What is Kubernetes Container Runtime?
A Container Runtime is the software responsible for running containers inside Kubernetes nodes.

Kubernetes itself does not directly create or execute containers. The Kubelet communicates with the container runtime through the Container Runtime Interface, or CRI.
The runtime handles tasks such as:
Pulling container images
Creating containers
Starting containers
Stopping containers
Managing container lifecycle
Modern Kubernetes commonly uses runtimes such as containerd and CRI-O.
EXAMPLE:
Suppose our e-commerce backend uses this image:
backend:v2
Kubernetes needs to run this image on a worker node.
The flow is:
Kubelet
↓
Container Runtime
↓
Pull backend:v2
↓
Create Container
↓
Start Container
Suppose the image is not available locally.
The runtime pulls it from a container registry and then starts the container.
Simple flow:
Kubernetes
↓
Kubelet
↓
CRI
↓
Container Runtime
↓
Container
↓
Application
Main point:
Container Runtime = Software that actually creates and runs containers on Kubernetes nodes.
;;;
63. What is Kubernetes Kube-proxy?
Kube-proxy is a networking component that runs on Kubernetes nodes and helps implement Service networking by maintaining network rules that allow traffic destined for Services to reach the appropriate backend Pods.
Suppose our e-commerce application has:
Backend Pod 1
Backend Pod 2
Backend Pod 3
We create:
backend-service
Users or other applications communicate with the Service rather than directly connecting to individual Pod IP addresses.
The Service traffic is handled using networking mechanisms configured on the node, commonly through technologies such as iptables or IPVS depending on the environment and configuration.
EXAMPLE:
Frontend Pod
↓

backend-service
↓
Kubernetes Service
↓
Backend Pod 1 / Pod 2 / Pod 3

If the request reaches Worker Node 1, the networking rules on the node help direct the Service traffic toward an appropriate backend Pod.

Simple flow:
Client
↓
Service IP
↓
Node Networking Rules
↓
Backend Pod

Main point:
Kube-proxy = Helps implement Kubernetes Service networking and route Service traffic toward Pods.

;;;

64. What is Kubernetes Pod Lifecycle?

Pod Lifecycle describes the different phases and states a Pod can go through from creation until it finishes or is removed.

The main Pod phases are:
Pending
Running
Succeeded
Failed
Unknown

EXAMPLE:
Suppose we deploy our e-commerce backend.

Initially:
Backend Pod
↓
Pending

The Pod is waiting for scheduling or for required resources/images to become available.

After it is scheduled and its containers start:
Backend Pod
↓
Running

If the Pod performs a finite task such as a Job and completes successfully:
Running
↓
Succeeded

If the task fails:
Running
↓
Failed

For example:
Pod created
↓
Pending
↓
Scheduled
↓
Container starts
↓
Running
↓

Application completes
↓
Succeeded

For continuously running applications such as a backend Deployment, the Pod normally remains in the Running phase while its containers are running.

Main point:

Pod Lifecycle = Describes the major phases a Pod goes through from creation to completion or failure.

;;;

65. What is Kubernetes Pod Restart Policy?

Restart Policy defines how Kubernetes should handle containers that terminate inside a Pod.

The main restart policies are:

Always
OnFailure
Never

For Pods managed by Deployments, the restart policy is normally Always.

EXAMPLE:

Suppose our backend Pod is running:

Backend Container
↓
Application Running

Now the application crashes.

With:

restartPolicy: Always

Kubernetes can restart the container.

Flow:

Container Running
↓
Application Crash
↓
Kubelet detects failure
↓
Container Restart
↓
Application Running

For a Job, OnFailure or Never can be used depending on the desired behavior because Jobs are intended to complete.

For example:

Job
↓
Task fails
↓
OnFailure
↓
Container can be restarted

Main point:

Restart Policy = Defines when containers inside a Pod should be restarted after termination.

;;;

66. What is Kubernetes EndpointSlice?

EndpointSlice is a Kubernetes resource that tracks the network endpoints associated with a Service.

When a Service selects Pods, Kubernetes needs to maintain information about which Pod IP addresses and ports are currently available as endpoints.

EndpointSlice provides this endpoint information in a scalable way.

EXAMPLE:

Suppose our e-commerce backend has:

Backend Pod 1 → 10.0.1.10
Backend Pod 2 → 10.0.1.11
Backend Pod 3 → 10.0.1.12

Our Service is:
backend-service
Kubernetes maintains endpoint information for the Service.
Backend Service
↓
EndpointSlice
↙ ↓ ↘
Pod 1 Pod 2 Pod 3
Suppose Pod 2 is deleted.
Kubernetes updates the endpoint information so traffic is directed only toward currently appropriate endpoints.
This is especially important in large clusters where a Service may have many endpoints.
Simple flow:
Service
↓
EndpointSlice
↓
Pod IPs / Ports
↓
Backend Pods
Main point:
EndpointSlice = Scalable way Kubernetes tracks the network endpoints behind a Service.
;;

67. What is Kubernetes Pod Disruption Budget?
Pod Disruption Budget, or PDB, is used to limit how many replicas of an application can be voluntarily disrupted at the same time.
This helps maintain application availability during operations such as node maintenance or cluster upgrades.
EXAMPLE:
Suppose our e-commerce backend has:
Backend → 5 replicas
We create a PDB that requires:
minimum available = 4
Now suppose Kubernetes needs to perform voluntary maintenance on nodes.
It should avoid voluntarily disrupting too many backend Pods at once so that the required availability is maintained.
For example:
5 Pods Running
↓
Maintenance
↓
1 Pod disrupted
↓
4 Pods remain available
But disrupting another Pod could violate the configured budget.
Important point:
PDB mainly protects against voluntary disruptions. It does not prevent failures such as a node suddenly crashing.
Simple flow:
5 Backend Pods
↓
PDB
↓
Minimum availability maintained
↓
Node Maintenance
Main point:

PDB = Protects application availability during voluntary Pod disruptions.
;;;

68. What is Kubernetes ResourceQuota?

ResourceQuota is used to limit the total amount of resources or number of certain objects that can be consumed within a Namespace.

It is useful when multiple teams or environments share the same Kubernetes cluster.

EXAMPLE:

Suppose our cluster has:
development namespace
We don't want developers to consume unlimited resources.
We can configure a ResourceQuota such as:
CPU → 10 cores
Memory → 20Gi
Pods → 20
Now the namespace cannot create resources beyond the configured quota.
For example:
Development Namespace
↓
ResourceQuota
↓
Maximum CPU = 10 CPU
Maximum Memory = 20Gi
Maximum Pods = 20
This prevents one team from consuming the entire cluster capacity.
Simple flow:
Namespace
↓
ResourceQuota
↓
Resource Limits
↓
Pods / Containers
Main point:
ResourceQuota = Limits the total resource and object consumption inside a Namespace.
;;;

69. What is Kubernetes LimitRange?

LimitRange is used to define default, minimum, and maximum resource constraints for containers or Pods within a Namespace.

ResourceQuota controls the total namespace consumption, while LimitRange controls resource settings for individual containers or Pods.

EXAMPLE:

Suppose developers create a Pod without specifying CPU and memory.
We can configure a LimitRange:
Default CPU → 500m
Default Memory → 512Mi
Now if a container does not specify these values, Kubernetes can apply the configured defaults.
We can also define minimum and maximum allowed resources.
For example:
Container
↓
LimitRange
↓
Minimum CPU = 100m
Maximum CPU = 2 CPU
This prevents developers from creating containers with inappropriate resource values.
Simple comparison:
LimitRange → Individual Pod / Container limits and defaults

ResourceQuota → Total Namespace resource consumption
Main point:
LimitRange = Defines default, minimum, and maximum resource constraints for individual workloads in a Namespace.
;;;
70. What is Kubernetes Admission Controller?
An Admission Controller is a component that intercepts requests to the Kubernetes API Server after authentication and authorization but before the object is persisted.
Admission controllers can validate or modify requests according to cluster policies.
EXAMPLE:
Suppose a developer tries to create a Pod.
The request flow is:
Developer
↓
API Server
↓
Authentication
↓
Authorization
↓
Admission Control
↓
etcd
Suppose the cluster has a policy:
"Containers must not run as privileged."
An admission controller can validate the request and reject it if the Pod violates the policy.
Admission control can also modify objects.
For example:
Developer creates Pod
↓
Admission Controller
↓
Adds required configuration
↓
Pod accepted
There are different types of admission controllers, including validating and mutating admission mechanisms.
Simple flow:
kubectl
↓
API Server
↓
Authentication
↓
Authorization
↓
Admission
↓
Object Stored
Main point:
Admission Controller = Validates or modifies Kubernetes API requests before resources are persisted.
;;;
71. What is Kubernetes Authentication vs Authorization?
Authentication and Authorization are two different security steps in Kubernetes.
Authentication answers:
"Who are you?"
Authorization answers:

"What are you allowed to do?"
EXAMPLE:
Suppose a developer runs:
kubectl get pods
First Kubernetes needs to identify the user.
Authentication:
Who is this user?
↓
Developer
After identifying the user, Kubernetes checks permissions.
Authorization:
Can this developer get Pods?
If allowed:
Request → Allowed
If not allowed:
Request → Forbidden
Simple flow:
User
↓
Authentication
↓
Who are you?
↓
Authorization
↓
What can you do?
↓
Allow / Deny
RBAC is commonly used for authorization.
Main point:
Authentication = Identifies the user.
Authorization = Determines what that user is allowed to do.
;;;
72. What is Kubernetes Admission Webhook?
An Admission Webhook allows an external service to participate in Kubernetes admission processing.
There are two common types:
Mutating Admission Webhook
Validating Admission Webhook
A Mutating Webhook can modify an incoming resource.
A Validating Webhook can accept or reject an incoming resource.
EXAMPLE:
Suppose our organization has a security policy:
"Every production Pod must contain specific security settings."
A developer creates:
Production Pod
The API Server sends the admission request to the configured webhook.
Webhook checks the Pod.
If the Pod is invalid:
Request → Rejected
If the webhook is mutating, it may modify the object before it is stored.
Simple flow:
kubectl
↓
API Server
↓
Admission Webhook
↓

Validate / Modify

↓

Accept / Reject

Main point:

Admission Webhook = Allows external logic to validate or modify Kubernetes API requests.

;;

73. What is Kubernetes Rolling Back a Deployment?

Rollback means returning a Deployment to a previous working revision when a newly deployed version has a problem.

EXAMPLE:

Suppose our e-commerce backend is running:

Backend v1 → Stable

We deploy:

Backend v2

But after deployment, users start receiving errors.

We can check the rollout history and rollback to the previous revision.

For example:

Backend v1

↓

Deploy v2

↓

Application Error

↓

Rollback

↓

Backend v1

A common command is:

```
kubectl rollout undo deployment/backend
```

We can check rollout history using:

```
kubectl rollout history deployment/backend
```

This is useful when a new deployment introduces application issues.

Simple flow:

v1 → Stable

↓

Deploy v2

↓

Problem

↓

Rollback

↓

v1 Restored

Main point:

Deployment Rollback = Reverts a Deployment to a previous revision when the new version has problems.

;;

74. What is Kubernetes Service Discovery?

Service Discovery is the mechanism through which applications find and communicate with Kubernetes Services.

Pods are dynamic and their IP addresses can change.

Instead of hard-coding Pod IP addresses, applications normally communicate using the Kubernetes Service name.

EXAMPLE:

Suppose we have:

Frontend Pod

Backend Service

Backend Pods

Frontend does not need to know:

Backend Pod 1 IP
Backend Pod 2 IP
Backend Pod 3 IP
Instead it can communicate using:
backend-service
Kubernetes DNS resolves the Service name and traffic is sent to the appropriate backend endpoints.
Flow:
Frontend
↓
backend-service
↓
Kubernetes DNS
↓
Backend Service
↓
Backend Pods
This makes application communication more stable.
Main point:
Service Discovery = Allows applications to find Kubernetes Services using stable names instead of changing Pod IP addresses.
;;
75. What is Kubernetes Horizontal Pod Autoscaler vs Cluster Autoscaler?
HPA and Cluster Autoscaler both provide scaling, but they scale different things.
HPA scales Pods.
Cluster Autoscaler scales Nodes.
EXAMPLE:
Suppose our e-commerce application currently has:
3 Worker Nodes
And:
Backend → 4 Pods
During a sale, traffic increases.
HPA detects increased CPU or another configured metric.
4 Pods
↓
HPA
↓
8 Pods
Now suppose the existing 3 worker nodes do not have enough capacity for the additional Pods.
Some Pods may remain Pending.
Cluster Autoscaler can then add worker nodes:
3 Nodes
↓
Insufficient Capacity
↓
Cluster Autoscaler
↓
5 Nodes
So both can work together.
Simple flow:
High Traffic
↓
HPA
↓
More Pods
↓
Not Enough Node Capacity
↓

Cluster Autoscaler
↓
More Nodes
Main point:
HPA = Scales Pods.
Cluster Autoscaler = Scales Worker Nodes.